

Feedback Project

A comprehensive, role-based academic feedback system built with **Flask**, **SQLAlchemy**, and **MySQL**. This application facilitates anonymous student feedback collection, teacher evaluation, and administrative reporting across four distinct roles: **Admin**, **HOD**, **Teacher**, and **Student**.

Features

Multi-Role Authentication

- Dedicated login portals for **Admins/Teachers/Students** and a separate **Management Portal** for HODs.
- Session-based authentication with role-specific route protection.

Anonymous Feedback System

- **Token-based submissions** ensure complete student anonymity — feedback cannot be traced back to individuals.
- Dynamically generated forms based on student class, division, batch, and elective subjects.
- Separate configurable question sets for **Theory** and **Practical** subjects (stored as JSON).

Admin Panel

- Manage **Teachers**, **Subjects**, **Sessions**, and **Allocations** (teacher → subject → class mappings).
- **Bulk CSV upload** for importing students, teachers, and subjects.
- Assign **Class Teachers** and **promote/rollback** students across semesters.
- **Semester-wise elective configuration** — set the number of electives per semester.
- View detailed feedback **reports** with per-question breakdowns and subjective comments.
- **Dashboard** with live counts of total teachers and students.

HOD (Head of Department) Portal

- Create and manage **Admin** accounts.
- Review and **approve/reject** feedback reports submitted by teachers.
- Access consolidated feedback results across all sessions and teachers.
- **Dashboard** with live counts of total teachers and students.

Teacher Portal

- View personal feedback scores and student comments.
- Manage student records: add, edit, delete students and assign elective subjects.

- **PRN field is read-only** during student edits to prevent accidental changes.
- **Acknowledge** feedback reports for HOD review.

Student Portal

- Generate a one-time anonymous **feedback token**.
- Submit feedback for all allocated subjects in a single, guided form.

Automated Backups

- **Daily SQL backups** via mysqldump with automatic scheduling on production server startup.
- Backups stored in the backups/ directory with date-stamped filenames (backup_YYYY-MM-DD.sql).
- **7-day rolling retention** — older backups are automatically deleted.
- Can also be run manually: `python backup.py`.

Security

- **Global CSRF protection** via Flask-WTF on all forms and AJAX endpoints.
- **PBKDF2-SHA256** password hashing using Werkzeug.
- **Admin password confirmation** required for destructive actions (deleting teachers/subjects).
- **Read-only PRN/EmpID fields** in edit forms to prevent accidental identity changes.
- Role-based route locking prevents unauthorized access.

Prerequisites

Requirement	Version
Python	3.8+
MySQL Server	5.7+ (XAMPP/WAMP or native)
pip	Latest recommended

Installation & Setup

1. Clone the Repository

```
git clone <your-repository-url>
cd FeedBack_Project
```

2. Create a Virtual Environment (Recommended)

```
python -m venv venv
```

On Windows:

```
venv\Scripts\activate
```

```
# On Mac/Linux:  
source venv/bin/activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Configure Environment Variables

Create a `.env` file in the project root:

```
DB_HOST=localhost  
DB_USERNAME=root  
DB_PASSWORD=your_mysql_password  
DB_NAME=feedback_db  
SECRET_KEY=generate_a_strong_random_secret_key
```

[!NOTE] If your local MySQL doesn't use a password, leave `DB_PASSWORD=` blank.

The app will **automatically create the database and tables** on first startup.

Usage

Development Server

For testing and development with hot-reload:

```
python run.py
```

Access at: `http://127.0.0.1:5000`

Production Server

For deployment or allowing concurrent access over your local network:

```
python serve.py
```

Access at: `http://0.0.0.0:80` (falls back to port 5000 if port 80 is unavailable).

[!TIP] Share your machine's IP address (e.g., `http://192.168.x.x`) with students so they can access the feedback portal from their devices.

The production server will also: - **Run a database backup on startup.** - **Schedule daily automatic backups** (every 24 hours) in the background.

Manual Backup

To trigger a database backup manually at any time:

```
python backup.py
```

Backups are saved to the `backups/` directory. Only the last 7 days of backups are retained.

Getting Started — First-Time Setup

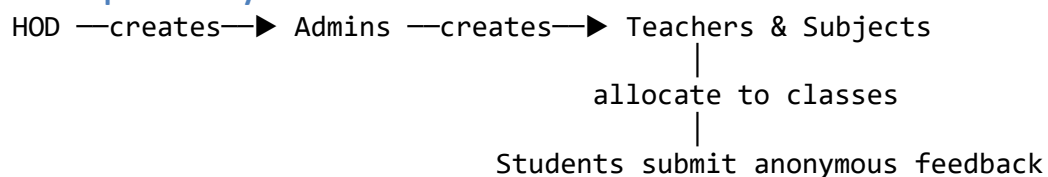
1. Root HOD Account (Auto-Created)

The root HOD account is **automatically created** the first time you start the server (python run.py or python serve.py). No manual setup is needed.

Default HOD credentials: | Field | Value | |—|—| | ID | HOD101 | | Password | HOD@123 |

[!IMPORTANT] Change the default HOD password after your first login. To customize the initial credentials, edit create_hod.py before the first server start.

2. Setup Hierarchy



3. Important Routes

Route	Portal	Who Can Access
/login	Main Portal	Admins, Teachers, Students
/management_login	Management Portal	HODs only

Project Structure

```
FeedBack_Project/
├── app/
│   ├── __init__.py           # App factory, CSRF setup, blueprint
│   ├── registration
│   │   ├── models.py         # 14 SQLAlchemy models (User, Session,
│   │   │                   Subject, etc.)
│   │   └── utils.py          # Helper to load feedback questions from
│   └── JSON
│       ├── data/
│       │   ├── theory_questions.json    # Configurable theory feedback questions
│       │   └── lab_questions.json       # Configurable practical feedback
│       └── questions
│           ├── routes/
│           │   ├── auth_routes.py        # Login/logout for all roles
│           │   ├── admin_routes.py       # Full admin CRUD & reporting
│           │   ├── hod_routes.py         # HOD management & report approval
│           │   ├── student_routes.py     # Token generation & feedback submission
│           │   └── teacher_routes.py     # Student management & report viewing
│           └── templates/
│               ├── login.html            # Main login page
│               └── management_login.html # HOD login page
```

— student.html	# Student dashboard
— feedback_form.html	# Anonymous feedback form
— admin/ (16 templates)	# Admin panel views
— hod/ (9 templates)	# HOD portal views
— teacher/(8 templates)	# Teacher portal views
— config.py	# Environment & database configuration
— create_hod.py	# One-time HOD account creation script
— backup.py	# Automated daily SQL backup script
— run.py	# Development server entry point (Flask)
— serve.py	# Production server entry point (Waitress +
TransLogger + Scheduled Backups)	
— requirements.txt	# Python dependencies
— backups/	# Auto-generated SQL backup files (7-day
retention)	
— logs/	# Server log files
— .env	# Environment variables (not committed)
— .gitignore	
— *.csv	# Sample CSV files for bulk uploads

Database Schema

The application uses **14 interconnected models**:

Model	Purpose
User	All users (admin, teacher, student, HOD) with role-based access
Session	Feedback collection sessions (open/closed status)
Subject	Theory/Practical subjects with elective & linked-subject support
StudentElective	Maps students to their chosen elective subjects
Allocation	Maps teacher → subject → target class (semester/division/batch)
TokenLog	Tracks whether a student has generated/submitted a token per session
ValidToken	Pool of valid anonymous tokens
ActiveTokenMap	Links students to their currently active token (session-scoped)
FeedbackResult	Individual question ratings per allocation
FeedbackComment	Subjective text comments per allocation
ClassTeacherAllocation	Assigns teachers as class teachers for specific divisions

Model	Purpose
ReportApproval	Tracks teacher acknowledgment and HOD approval of feedback reports
SemesterConfig	Configures the number of elective subjects per semester

Tech Stack

Layer	Technology
Backend	Flask, Flask-SQLAlchemy, Flask-WTF
Database	MySQL (via PyMySQL)
Auth	Werkzeug (PBKDF2-SHA256), Flask sessions
WSGI Server	Waitress (production), Flask dev server
Logging	Paste TransLogger (production access logs)
Backups	mysqldump (automated via backup.py)
Templating	Jinja2

Security Notes

[!CAUTION] **Never commit .env to version control.** It contains your database credentials and secret key.

- **Default Teacher Passwords:** When Admins create Teachers, the default password is {EmpID}@123 (e.g., T101@123). Users should change this on first login.
- **Destructive Actions:** Deleting subjects or teachers **cascades** to delete all associated feedback data. These actions require **Admin password confirmation**.
- **CSRF Protection:** All form submissions and AJAX requests are protected by CSRF tokens.
- **Read-Only Identifiers:** PRN (students) and EmpID (teachers) fields are locked during edits to preserve data integrity.

License

This project is for academic/educational use.